Test Answers and Grader Responses-

Test Answers Plan –

Test answers were written by Claude Code.

On the 4 test answers

I can generate those right now — no setup needed. I'd write them at four levels:

- **Strong PASS** — hits all 3 concepts clearly, uses enterprise language naturally
- **Weak PASS** — gets all 3 but the SOX audit trail piece is thin/implicit
- **PARTIAL** — solid on hooks but misses the audit trail entirely, or gets audit trail but thinks better prompting is part of the fix
- **FAIL** — developer mindset, focuses on prompt wording or adding logging calls

We can analyze what the rubric *should* produce for each, then once you're live, run them through the actual grader and see if Claude's output matches expectations. That's your real test of whether the framework holds up.

Test Answers and Grader Response

Answers are written as realistic exam responses — the kind of range you'd actually see from people with varying backgrounds.

Answer 1 — Strong PASS

The core architectural failure is that this system relies entirely on system prompt instructions to enforce a financial control. That is not a control — it is a suggestion. Language model behavior is probabilistic. In a SOX-regulated environment, any control with a non-zero failure rate is an audit finding waiting to happen. The $50,000 approval threshold is a material internal control under SOX Section 302/404. It cannot live in a prompt.

The fix requires PostToolUse hooks on the process_payment tool. The hook intercepts every call to that tool before execution, reads the invoice amount, checks whether a documented manager approval exists in the authorization record, and blocks execution if the threshold is exceeded without it. The agent's instructions become irrelevant to enforcement — the hook either permits or denies the call regardless of what the prompt says.

The second issue the Finance Director is almost certainly flagging is the audit trail. SOX doesn't just require that controls exist — it requires documented evidence that they fired on specific transactions. A write_audit_log tool call that the agent decides to make is not sufficient. The hook must generate its own tamper-evident log entry: timestamped, recording the tool call parameters, the authorization check result, and the permit/block decision. This record needs to exist independent of the agent's behavior, because the agent's behavior is exactly what auditors are verifying.

Putting process_payment in the tools list with no hook is the functional equivalent of telling your AP clerk "please remember to get approval for large payments" and calling that your internal control.

Grader response-

PASS Textbook-quality response demonstrating mastery of all three rubric concepts with SOX-specific precision.

Graded against SOX compliance rubric · Task 1.5 · Finance vertical

CONCEPTS DEMONSTRATED

- ✓ PROGRAMMATIC ENFORCEMENT GAP: Explicitly and correctly identifies that prompt-based enforcement is probabilistic and therefore not a valid SOX control — 'a control with a non-zero failure rate is an audit finding waiting to happen'
- ✓ HOOK-BASED SOLUTION: Correctly specifies PostToolUse hooks on process_payment, describes the interception logic (read amount, check authorization record, permit/block), and correctly notes that hooks enforce independently of prompt instructions
- ✓ SOX AUDIT TRAIL REQUIREMENT: Correctly distinguishes between agent-generated log calls (insufficient) and hook-generated tamper-evident records (required), and explains why — auditors are verifying the agent's behavior, so the log cannot depend on that behavior

CONCEPTS MISSING

None — complete answer

KEY STRENGTH

The candidate's insight that a write_audit_log tool call the agent decides to make is not sufficient — and the reasoning behind it — is the most sophisticated point in the response. Many candidates propose adding a logging tool and consider the audit trail solved. This candidate correctly identifies that agent-dependent logging creates a circular dependency: you cannot use the thing being audited to generate the audit evidence. This is exactly the reasoning a SOX auditor would apply.

KEY GAP

No material gaps. Minor observation: the candidate mentions PostToolUse hooks but the more precise fit for blocking execution before it occurs would be a pre-execution or PreToolUse hook. PostToolUse hooks fire after execution completes, which means a payment could process before the hook runs — a meaningful distinction in a payment context. This does not affect the grade but would be worth addressing in a production design.

EXAMINER NOTE

The closing analogy — equating an unhoooked process_payment tool to telling an AP clerk to 'please remember' approval — demonstrates that the candidate understands not just the technical pattern but why it matters under SOX's control framework. For production hardening, the candidate should revisit whether PostToolUse is the correct hook phase for blocking payments, as pre-execution interception is necessary to prevent the payment from processing before the authorization check occurs. The candidate should also consider specifying that the authorization record itself must be immutable and timestamped, not just the hook log, to close the chain of custody from approval through execution.

Answer 2 — Weak PASS

The problem is that the approval requirement for payments over $50,000 is written into the system prompt as an instruction, not enforced programmatically. LLMs don't guarantee that they follow instructions 100% of the time — there's always some probability of the instruction being ignored, misinterpreted, or bypassed in an edge case. For a financial control that needs to be reliable, that's not acceptable.

The right approach is to implement hooks in the Agent SDK that intercept the process_payment tool call. Before the payment executes, the hook should check whether manager approval has been documented for that invoice. If not, the hook blocks the call and routes it to the approval workflow. This makes the control deterministic — it doesn't matter what the agent was told to do, the hook enforces the rule at the infrastructure level.

For SOX compliance you also need logging. Every time the hook fires, it should record what happened — which invoice, what amount, whether it was approved or blocked, and when. This gives auditors the evidence they need to verify the control is working. The system currently relies on the agent calling write_audit_log itself, which isn't reliable enough because it's still just an instruction.

Grader response-

PASS Candidate demonstrates clear, accurate understanding of all three rubric concepts with precise reasoning grounded in the SOX control environment.

Graded against SOX compliance rubric · Task 1.5 · Finance vertical

CONCEPTS DEMONSTRATED

- ✓ PROGRAMMATIC ENFORCEMENT GAP: Explicitly identifies that prompt-based instructions carry non-zero failure probability and that this is architecturally unacceptable for a financial control — not just suboptimal, but a compliance gap.
- ✓ HOOK-BASED SOLUTION: Correctly describes pre-execution hook interception of process_payment, programmatic verification of documented approval, and deterministic blocking when approval is absent — including the key insight that the control fires at infrastructure level regardless of agent behavior.
- ✓ SOX AUDIT TRAIL REQUIREMENT: Identifies that agent-self-reported audit logging (write_audit_log as an instruction) is insufficient, and correctly frames the solution as hook-generated logs that provide auditor-verifiable evidence that the control actually fired.

CONCEPTS MISSING

None — complete answer

KEY STRENGTH

The candidate's observation that write_audit_log being agent-invoked makes it 'just another instruction' is particularly sharp — this closes a common loophole where architects implement hooks for enforcement but leave logging vulnerable to the same prompt-compliance failure they were trying to eliminate. Auditors would catch exactly this gap.

KEY GAP

No material gaps relative to the rubric. The response could be strengthened by noting tamper-evidence properties of hook logs (e.g., logs written outside agent control to an immutable store), but absence of this detail does not constitute a rubric failure.

EXAMINER NOTE

This response would hold up in a SOX audit readiness review. The candidate correctly frames the control as needing to be deterministic and independently verifiable — both of which are core to what auditors mean when they test whether a control 'exists' versus whether it 'works.' In a real implementation, the candidate should ensure hook logs are written to an append-only system outside the agent's write scope, since a compromised or hallucinating agent that can write to its own audit log undermines the independence of the control. That's the natural next design consideration from where this answer ends.

Answer 3 — PARTIAL

The architecture has two problems. First, the system prompt instructions are the only thing enforcing the approval requirement, and they're not specific enough. The prompt says "you MUST require manager approval" but doesn't define what counts as approval, how to verify it, or what to do if the manager doesn't respond. The agent could interpret this in different ways.

Second, and more importantly, there are no hooks configured. The Agent SDK has a hook system specifically designed for this kind of workflow enforcement. A pre-execution hook on the process_payment tool would intercept the call before it runs, verify that the invoice amount has been approved, and block it if not. This is much more reliable than relying on the agent to make the right decision based on prompt instructions alone.

To fix this: implement hooks on process_payment, strengthen the prompt language to be more explicit about the approval verification steps, and make sure the notify_manager tool is called before process_payment in any workflow involving amounts over the threshold. The Finance Director is right that this needs to be tightened up before it can go to a SOX audit.

Grader response-

PARTIAL

Candidate demonstrates strong understanding of the programmatic enforcement gap and hook-based solution but fails to address the SOX audit trail requirement, which is a material omission for a regulated environment.

Graded against SOX compliance rubric · Task 1.5 · Finance vertical

CONCEPTS DEMONSTRATED

✓ **PROGRAMMATIC ENFORCEMENT GAP:** Clearly articulates that prompt-only enforcement is unreliable and that the agent could 'interpret this in different ways' — the core insight that a probabilistic control is not a control.

✓ **HOOK-BASED SOLUTION:** Correctly identifies pre-execution hooks on process_payment as the right mechanism, describes intercepting the call before execution, verifying authorization, and blocking on failure — all accurate.

CONCEPTS MISSING

✗ **SOX AUDIT TRAIL REQUIREMENT:** The candidate never addresses what the hook must produce as evidence. There is no mention of timestamped logs, tamper-evident records, or the distinction between 'the control was instructed to fire' and 'the control demonstrably fired.' The candidate says this needs to be ready for a SOX audit but does not explain what SOX auditors actually require from the hook implementation.

KEY STRENGTH

The candidate's identification of the prompt enforcement gap is precise and well-reasoned — correctly noting that ambiguity in approval definition creates interpretive risk, and correctly locating the fix in the hook layer rather than prompt refinement.

KEY GAP

No discussion of audit trail quality from the hooks. A SOX auditor will not accept 'we have a hook that blocks unapproved payments' — they require documented, immutable evidence that the control fired on specific transactions, with timestamps and outcomes. Without this, the architecture still fails a SOX audit even with hooks in place.

EXAMINER NOTE

The recommendation to 'strengthen the prompt language' alongside the hook fix slightly undercuts the candidate's own correct insight — in SOX context, the prompt is irrelevant to control validity once the hook is in place. More critically, the candidate should specify that hook logs must be written to an append-only audit store with sufficient metadata (transaction ID, invoice amount, authorization reference, control decision, timestamp) to satisfy SOX Section 404 evidence requirements. The phrase 'ready for a SOX audit' at the end suggests awareness of the regulatory stakes but the candidate never translates that awareness into concrete audit trail requirements.

Answer 4 — FAIL

The Finance Director's concern is valid. Looking at the code, there are several issues that would cause problems in a SOX audit.

The system prompt mentions the $50,000 threshold but the process_payment tool has no amount limit — the comment even says "Can execute payments up to any amount." This is a

direct contradiction. The tool should have validation built in to reject payments over the threshold unless an approval flag is set.

The audit log is also a concern. The agent is instructed to "always document your actions in the audit log" but this is vague. For SOX compliance, audit entries need to include specific fields — invoice ID, amount, approver name, timestamp, and approval date. The current instruction doesn't specify any of this.

I would recommend rewriting the system prompt to be much more specific: require the agent to call notify_manager and wait for a confirmation response before calling process_payment on any invoice over $50,000. Add validation parameters to the process_payment tool itself. And define a structured format for audit log entries that captures all the fields an auditor would need to see.

Grader response-

NOTE- PARTIAL vs FAIL for Answer 4: The grader gave partial credit for the tool validation observation and the audit log fields, which technically satisfied two concepts partially. The rubric needs a tighter definition — something like "a recommendation to fix the system prompt as the primary solution is disqualifying for concept 1 regardless of other observations." That's a rubric calibration note, not a system failure.

PARTIAL

The candidate identifies the tool-level validation gap and audit logging deficiencies but does not recognize the architectural necessity of hook-based enforcement and treats the core problem as fixable through better prompt writing.

Graded against SOX compliance rubric · Task 1.5 · Finance vertical

CONCEPTS DEMONSTRATED

✓ PROGRAMMATIC ENFORCEMENT GAP (partial): The candidate correctly identifies that the process_payment tool has no amount-based enforcement and that the system prompt alone cannot substitute for built-in validation — 'The tool should have validation built in to reject payments over the threshold unless an approval flag is set' shows awareness that code-level enforcement is needed.

✓ SOX AUDIT TRAIL REQUIREMENT (partial): The candidate correctly flags that the current audit logging is vague and specifies the structured fields an auditor would require — invoice ID, amount, approver name, timestamp, approval date.

CONCEPTS MISSING

✗ HOOK-BASED SOLUTION: The candidate never mentions Agent SDK hooks, pre-execution interception, or any mechanism that programmatically intercepts the tool call before it fires. The recommended fix is to rewrite the system prompt and add parameters to the tool — both of which are still prompt/configuration-layer solutions, not deterministic control enforcement.

✗ PROGRAMMATIC ENFORCEMENT GAP (fully): While the candidate gestures at tool-level validation, the final recommendation circles back to 'rewriting the system prompt to be much more specific' as the primary fix. This actively contradicts the rubric requirement — a candidate who recommends a more detailed prompt as the solution has demonstrated they believe prompt-based compliance is architecturally sufficient, which it is not under SOX.

KEY STRENGTH

The candidate's identification of the audit log gap is practical and audit-ready — naming the specific fields (invoice ID, approver name, approval date) reflects genuine understanding of what SOX documentation requirements look like in practice.

KEY GAP

The candidate's recommended fix relies on a better system prompt as the enforcement mechanism. In a SOX context, this is the core architectural failure the question is testing: a prompt instruction that 'require[s] the agent to call notify_manager and wait for a confirmation response' is still a behavioral instruction, not a control. An LLM can fail to follow it, and an auditor will correctly reject it as unverifiable. The candidate needed to identify that a hook — code that intercepts the tool call, checks authorization state, and blocks execution if criteria are unmet — is what transforms a behavioral guideline into a documented, deterministic control.

EXAMINER NOTE

For SOX purposes, the distinction between 'the agent is instructed to check approval' and 'the agent is blocked from proceeding without approval' is not semantic — it is the difference between a compensating control and an actual control. A pre-execution hook that queries an authorization record and throws if it is absent will produce a log entry proving the control fired on every transaction; a system prompt will not. The candidate should study how Agent SDK lifecycle hooks (PreToolUse/PostToolUse) can be used to enforce approval gates programmatically and how those hook execution logs satisfy the SOX requirement for evidence that controls actually operated — not just that they were instructed to operate.

What I expect the grader to do with these:

Answer	Expected	Reasoning
1	PASS	All 3 concepts, strong SOX framing, "tamper-evident" language
2	PASS	All 3 present — audit trail is thin but the concept is there

Answer	Expected	Reasoning
3	PARTIAL	Hooks + enforcement gap are solid; audit trail is missing; the "strengthen the prompt" line is actively wrong
4	FAIL	Prompt rewriting and tool validation — never grasps why the prompt itself is the problem

The interesting one to watch is Answer 2 — if the grader is lenient it's a PASS, if strict it might land PARTIAL. That will tell you something real about the rubric calibration.